

Technische handleiding & data-recovery

Voor technisch beheer · Intern Studentbeheersysteem (SIS) · IUASR

Dit document is bestemd voor **technisch personeel/beheerders**. Het beschrijft de architectuur, het maken van back-ups en de **herstelprocedure**. Bewaar het vertrouwelijk.

0. Platform: modules

Het systeem is een multi-module platform. Na de login (route `modules.kiezen`, `/modules`) kiest de gebruiker een module. Tabel `modules` (sleutel, naam, actief, volgorde) is de registry; de vijf modules worden in de create-migratie ingevoegd. Nieuwe module toevoegen = één rij plus haar schermen, geen ingreep in de kern.

Moduletoegang volgt uit de rol: `RoL::moduleSleutels()` geeft de toegestane modulesleutels (`['*']` voor Beheerder). `Module::toegankelijkVoor()` toetst toegang, `bruikbaarVoor()` = toegankelijk én actief. Een module opent op `Module::startRoute()` (nu alleen Studentenzaken → dashboard). Nog niet gebouwde modules staan op `actief=false` en worden als 'Binnenkort' getoond. De rollen blijven de bestaande enum `App\Enums\RoL`; de cursus-specifieke rollen komen bij de Cursussen-module.

0b. Module Cursussen Administratie

Aparte administratie, lichter regime (geen BSN/DUO). Tabellen: cursussen (code, naam, cursusgeld, start/eind, directeur_id, actief; drie cursussen in de create-migratie), cursisten (aangepaste, lichtere kopie van studenten; cursistnummer = C + jaar + volgnummer via `CursistnummerGenerator`), cursusinschrijvingen (`cursist × cursus`, totaalbedrag als momentopname van het cursusgeld). Controllers onder `App\Http\Controllers\Cursus\*`, routes onder prefix `cursussen`. Rol `Cursusadministratie` toegevoegd aan de enum (enum-kolom via ALTER). Bulk-import leest .xlsx én .csv via `App\Support\Tabellezer` (PhpSpreadsheet), kolomherkenning op naam. Cursusdirecteuren met toegangsbeperking volgen in een latere fase.

Cursus kopiëren (Beheerder). `CursusController@kopieForm` op route `cursussen.kopieren` (`/cursussen/beheer/{bron}/kopieren`, rol:beheerder) rendert het bestaande `cursussen.form` vooraf ingevuld met een niet-persisted new `Cursus()` op basis van de bron (cursusgeld, omschrijving,

looptijd, directeur; code leeg). Opslaan verloopt via de reguliere store — er is dus geen aparte kopie-actie; de nieuwe cursus ontstaat uit de ingevulde velden. Inschrijvingen/cursisten worden NIET meegekopieerd. Let op de route-model-binding: de parameter heet bewust {bron} zodat hij bindt op `kopieForm(Cursus $bron)` (naam moet overeenkomen, anders krijgt de methode een leeg model).

Directe cursusknoppen op het welkomtscherm. `ModuleController@index` geeft de voor de gebruiker zichtbare, actieve cursussen mee (`Cursus::query()->zichtbaarVoor()`); `modules/index.blade.php` rendert ze als aparte tegels onder "Cursussen". Elke tegel linkt naar de cursus-startpagina `cursussen.cursus (/cursussen/cursus/{cursus}, CursusDashboardController@cursus, view cursussen.cursus)` in de dashboardgroep `rol:cursusadministratie,financien,beheerder,bestuur` met `abort_unless($cursus->zichtbaarVoor())`. De pagina toont per cursus de statusverdeling, financiële kerncijfers (`Cursusrapport::financieelTotaal`) en de cursisten, met snelkoppelingen naar cursusgelden (`?cursus=`) en rapportage. De naam-tegel toont bewust géén cursusgeld.

Cursusrapportage & dashboards (Fase E). Aggregatieklasse `App\Support\Cursusrapport`: per cursus de inschrijvingen (uitgesplitst per `CursusinschrijvingStatus`) en de cursusgelden (verschuldigd/betaald/openstaand + betaalgraad, via `Cursusgeldstatus`), plus totalen en een verdeling naar betaalmethode. Financiële cijfers tellen alleen niet-geannuleerde inschrijvingen; alleen betalingen met status `Betaald` gelden als voldaan. `Cursus\CursusrapportController` (`index = view cursussen.rapport, export = CSV op cursistniveau, gelogd als INZAGE veld cursusrapport_export`). Routes `cursussen.rapport + cursussen.rapport.export` in de dashboardgroep `rol:cursusadministratie,financien,beheerder,bestuur`, dezelfde `Cursus::query()->zichtbaarVoor()`-scoping als het dashboard (`directeur = eigen cursus[sen]`). Het cursusdashboard toont via `Cursusrapport::financieelTotaal()` de betaalgraad en het openstaande bedrag; de Bestuurspagina linkt door naar het rapport. Rendering met de bestaande chart-partialen. 8 tests (`CursusrapportTest`); 327 groen.

Cursusdirecteuren — toegang per cursus (Fase D). De rol `Cursusadministratie` is voortaan **cursusgebonden** (need-to-know), analoog aan de opleidinggebonden Directie. Sleutel is de bestaande FK `cursussen.directeur_id` (geen pivot nodig): een directeur dirigeert de cursussen waarvan hij `directeur_id` is (`User::gedirigeerdeCursussen()`, `User::cursusIds()`, `User::isCursusBeperkt()` → alleen `Cursusadministratie`). Scopes `Cursus::scopeZichtbaarVoor()` / `Cursist::scopeZichtbaarVoor()` (aanroepen via `::query()->zichtbaarVoor()` om botsing met de gelijknamige instance-guard te vermijden) + instance-guards `Cursus::zichtbaarVoor()/beheerbaarVoor()` en `Cursist::zichtbaarVoor()`. Toegepast in alle vier de admin-controllers: dashboard (drie tellingen gescoped), cursusbeheerlijst, cursistenlijst + `withCount`, cursus-dropdowns, en per-record guards op `edit/update/show/inschrijven`.

Rolverdeling in de routes: cursus aanmaken/verwijderen én directeur toewijzen is `rol:beheerder`

(server-side; in `CursusController::update` wordt `directeur_id` alleen toegepast als de gebruiker Beheerder is → geen rechtenescalatie). Cursusbeheer-details en cursisten/inschrijvingen muteren: `rol:cursusadministratie,beheerder` (gescoped). Boekhouding (betalingen): `rol:financien,beheerder,ongescored` (alle cursussen). Dashboard en cursisteninzage ook voor het **Schoolbestuur** (`Rol::moduleSleutels()` voor `Bestuur = ['studentenzaken', 'cursussen']`; `Rol::magCursusInzien()` = `Cursusadministratie/Beheerder/Bestuur`), alleen-lezen — de mutatieknoppen hangen in de views aan `magCursusBeheer()`. Seed (definitief, opdrachtgever): Arabische Taal → Hafsa; Hifz + Ijaaza → Omar (twee directeuren — Arabisch apart, Hifz en Ijaaza samen). Guarded datamigraties (`wijs_cursusdirecteuren_toe`, `herverdeel_cursusdirecteuren_een_per_cursus`) vullen/herverdelen een bestaande database (draaien op een verse migratie vóór de seeders en doen dan niets). 15 tests (`CursusdirecteurTest` + `tegentest`); 312 groen.

Cursusgelden & boekhouding (Fase C). Tabel `cursusbetalingen` (`cursusinschrijving_id` FK cascade, `betaalmethode`, `bedrag decimal(10,2)`, `betaaldatum`, `betalingsstatus`, `referentienummer`, `opmerking`, `geregistreerd_door_id` FK `users` nullOnDelete). Enums `App\Enums\Betaalmethode` (`ideal/overboeking/contant`) en `App\Enums\Cursusbetaalstatus` (`in_afwachting/betaald/mislukt/terugbetaald`; alleen `Betaald` telt als voldaan). De financiële status per inschrijving is **afgeleid, niet opgeslagen**: `App\Support\Cursusgeldstatus::voor()` zet totaalbedrag af tegen de som van de betalingen met status `Betaald` → {totaal, betaald, openstaand, status} (voldaan/deels/open). Controller `Cursus\CursusbetalingController` (`overzicht/registreer/bijwerken/verwijderen`); routes onder prefix `cursussen` met middleware `rol:financien,beheerder`. Het cursusdashboard staat op `rol:cursusadministratie,financien,beheerder` (de modulekiezer stuurt Financiën naar `cursussen.dashboard`); `cursus-/cursistbeheer` blijft `rol:cursusadministratie,beheerder`. Rolregels: `Rol::magCursusBeheer()` en `Rol::magCursusFinancien()` (met User-delegates) sturen de rolbewuste sidebar en dashboardknoppen. Wijzigen/verwijderen van betalingen wordt via `AuditLogger` gelogd (veld `cursusbetaling`).

Ob2. Module Relatiebeheer & Stagebeheer (Fase A)

Opzet. Opleidingoverstijgende module (PABO, Bachelor Islamitische Theologie, Master IGV) voor externe relaties. De placeholder-module stage is via de datamigratie `stage_module_naar_relatiebeheer` hernoemd naar sleutel `relatiebeheer` (naam "Relatiebeheer & Stage") en met `activeer_relatiebeheer_module` op `actief=true` gezet; `Module::START_ROUTES['relatiebeheer'] = 'relaties'`. Twee nieuwe rollen `Relatiebeheerder` en `Stagecoordinator` in `App\Enums\Rol` (enum-kolom via ALTER-migratie `add_relatiebeheer_rollen`, die `Rol::waarden()` leest — enum-case dus vóór de migratie). Ontwerp: `docs/MODULE-RELATIEBEHEER.md`.

Datamodel. `organisatie_types` (opzoektabel; code uniek, naam, `opleiding_id` nullable = per opleiding instelbaar of null=alle, actief), `organisaties` (relatienummer uniek/leesbaar via

App\Support\RelatienummerGenerator = R + jaar + volgnr, organisatie_type_id nullOnDelete, adres/contactvelden, actief, opmerkingen) en de pivot organisatie_opleidingen (echte FK's, cascade, unique). Modellen App\Models\Organisatie en OrganisatieType.

Scoping & rollen. Opleidinggebonden (need-to-know), analoog aan de Directie: de koppeling loopt via dezelfde User::opleidingen()-relatie (directie_opleidingen). Rol::isRelatieBeperkt() = Relatiebeheerder/Stagecoordinator/Directie; Organisatie::scopeZichtbaarVoor() filtert op whereHas('opleidingen', ... whereIn opleidingIds), plus instance-guards zichtbaarVoor()/beheerbaarVoor(). Rolregels: Rol::magRelatiebeheer() (Relatiebeheerder/Stagecoordinator/Beheerder = aanmaken/wijzigen), magStagebeheer() (Stagecoordinator/Beheerder — stageplaatsen, latere fase), magRelatieInzien() (+ Directie, Bestuur). moduleSleutels(): de twee nieuwe rollen → ['relatiebeheer'], Directie → ['studentenzaken', 'relatiebeheer'], Bestuur → ['studentenzaken', 'cursussen', 'relatiebeheer']. **Let op:** bij een nieuwe rol alle exhaustive match-methodes zonder default in Rol aanvullen (label, magCijfersInzien, magInschrijvingBeheren, magPresentieInzien, magAanwezigheidsregelingZien, moduleSleutels).

Controller/routes/views. App\Http\Controllers\Relatie\OrganisatieController (index met filters q/type/opleiding/status, create/store/show/edit/update/status-toggle). Routes onder prefix relatiebeheer: beheergroep rol:relatiebeheerder,stagecoordinator,beheerder (bewust vóór de inzagegroep zodat /organisaties/nieuw niet als id bindt), inzagegroep rol:relatiebeheerder,stagecoordinator,directie,bestuur,beheerder (relaties, relaties.show). Views resources/views/relaties/{index,form,show}; sidebar-tak \$inRelatiemodule = routeIs('relaties*'). Aanmaken/wijzigen gelogd via AuditLogger (veld organisatie/organisatie_status). Server-side afgedwongen dat een opleidinggebonden gebruiker alleen de **eigen** opleiding(en) kan koppelen (Rule::in(\$toegestaan) op opleidingen.*). **AVG-grens:** uitsluitend organisatie- en contactgegevens, nooit leerling-/cliëntgegevens. Landing-redirect (DashboardController) stuurt module-only rollen naar relaties. Startdata op de draaiende DB via guarded seed_relatiebeheer_startdata (no-op op verse test-DB). 10 tests (RelatiebeheerModuleTest); 361 groen.

Oc. Bestuurspagina & systeembeheer-snelkoppelingen

Globale bestuurspagina. Route /bestuur (bestuur, middleware rol:bestuur,beheerder), BestuurController@index → view bestuur.index (in de app-shell). Instellingsbreed, alleen-lezen; hergebruikt de bestaande App\Support\Statistiek-aggregaties (kern, perOpleiding, statusVerdeling, instroom, slaagpercentage, presentie(+Verdeling/PerOpleiding), financieel) plus cursuscijfers (Cursus/Cursist/Cursusinschrijving). Rendering met de chart-partials partials.charts.bar|spark|donut en de dashboardkaarten (sis-chartgrid/sis-chart-card, iuasr-dash-stat). Bereikbaar via een tegel op de modulekiezer en een menu-item in de sidebar (Bestuur + Beheerder).

Systeembeheer op de modulekiezer. `resources/views/modules/index.blade.php` toont voor `rol=beheerder` een blok met directe links naar gebruikers, opzoektabelen, audit-log, backup en `handleiding.technisch` (voorheen alleen via de Studentenzaken-sidebar). De routes zelf blijven in de `rol:beheerder-groep`; dit zijn uitsluitend snelkoppelingen op de hoofdpagina.

1. Architectuur & stack

- **Applicatie:** PHP + Laravel (server-gerenderd; geen kale PHP/WordPress).
- **Database:** MySQL/MariaDB (InnoDB, echte foreign keys, surrogaatsleutels).
- **Authenticatie:** Microsoft Entra ID (SSO/OIDC) — nooit een eigen login bouwen. In de ontwikkelomgeving een tijdelijke dev-login.
- **Netwerk:** draait intern (intranet), IP-beperkt, gescheiden van het publieke aanmeldportaal.
- **Gevoelige data:** BSN en rekeningnummer versleuteld (Laravel-encryptie met **APP_KEY**); inzage/mutatie gelogd (audit-log).

2. Omgeving (referentie)

PHP	<code>~/php/8.3/php.exe</code>
Composer	<code>~/bin/composer.phar</code>
Database-server	MariaDB (portable) — <code>~/mariadb/mariadb-11.4.9-winx64/bin/</code>
DB-host / poort	<code>127.0.0.1 : 3307</code>
Database / gebruiker	<code>iuasr_sis / iuasr_sis</code>
Configuratie	<code>.env</code> (in de projectmap) — bevat DB-gegevens en APP_KEY

APP_KEY is kritiek. Zonder de originele APP_KEY zijn de versleutelde velden (BSN, rekeningnummer) onherstelbaar. De sleutel zit in `.env` en wordt meegenomen in de back-up.

E-mail (resultaten mailen)

De examencommissie kan definitieve resultaten per e-mail naar studenten sturen. In **ontwikkeling** staat `MAIL_MAILER=log`: e-mails worden naar `storage/logs/laravel.log` geschreven, er gaan GEEN echte e-mails uit (AVG, synthetische data). In **productie** configureert u de IUASR-mailserver in `.env`:

```
MAIL_MAILER=smtp
MAIL_HOST=<smtp.iuasr.nl>
MAIL_PORT=587
```

```
MAIL_USERNAME=<gebruiker>
MAIL_PASSWORD=<wachtwoord>
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@iuasr.nl
MAIL_FROM_NAME="IUASR Studentenzaken"
```

Elke student ontvangt individueel de eigen (ondertekende) cijferlijst als bijlage; verzending wordt gelogd. Overweeg voor grote aantallen een queue (QUEUE_CONNECTION + worker).

3. Back-up maken

Een volledige back-up wordt gemaakt via de webapplicatie:

1. Log in als **Beheerder**.
2. Ga naar **Beheer → Back-up & herstel**.
3. Geef een sterk **wachtwoord** op (minimaal 8 tekens) en bevestig.
4. Klik op **Back-up genereren & downloaden**. U ontvangt een met AES-256 versleutelde ZIP.

De ZIP bevat: database.sql (volledige dump), de applicatiebroncode en webpagina's, .env (incl. APP_KEY) en de geüploade bestanden (storage/app). Niet inbegrepen: vendor/, .git/ en de referentiemap IUASR/. Het wachtwoord wordt **nergens opgeslagen**; downloaden wordt ge-audit-logd.

Advies: bewaar back-ups versleuteld op een beveiligde, interne locatie en hanteer een bewaarschema (bijv. wekelijks + vóór elke update). Overweeg een periodieke, geautomatiseerde back-up naar een netwerkschijf.

4. Herstelprocedure (recovery)

Terugzetten is een bewuste beheerhandeling en gebeurt **niet** vanuit de draaiende applicatie (die kan zichzelf niet veilig overschrijven en een database-restore wist bestaande data). Voer de stappen uit op de server.

Stap 1 — Archief uitpakken

De Windows Verkenner kan een AES-versleutelde ZIP **niet** openen. Gebruik het meegeleverde commando (of 7-Zip/WinRAR):

```
~/php/8.3/php.exe artisan backup:uitpakken "pad/naar/iuasr-sis-backup-JJJJMMDD-UUMM.zip" --doel="pad/naar/hersteld"
```

U wordt om het wachtwoord gevraagd (of geef `--wachtwoord=...`). Het commando verifieert het wachtwoord en pakt uit naar de doelmap.

Stap 2 — Bestanden plaatsen

Plaats de uitgekakte bestanden in de webroot/projectmap van de (interne) server. Zet ook `storage/app` (geüploade documenten) terug.

Stap 3 — Afhankelijkheden herstellen

```
~/bin/composer.phar install --no-dev --optimize-autoloader
```

Stap 4 — Database terugzetten

Maak (indien nodig) een lege database en importeer de dump. De dump bevat zowel het schema als de data (DROP/CREATE/INSERT):

```
~/mariadb/mariadb-11.4.9-winx64/bin/mariadb.exe -h 127.0.0.1 -P 3307 -u iuasr_sis -p  
iuasr_sis < database.sql
```

Let op: dit **overschrijft** de bestaande gegevens in de database `iuasr_sis`. Maak eerst een verse back-up van de huidige stand als die nog waarde heeft.

Stap 5 — Configuratie controleren

- Controleer `.env`: `DB_*`-gegevens en `APP_URL`.
- **Laat `APP_KEY` ongewijzigd** (gelijk aan de back-up), anders zijn BSN/rekeningnummer niet te ontsleutelen.

Stap 6 — Cache legen & controleren

```
~/php/8.3/php.exe artisan optimize:clear
```

Een `php artisan migrate` is **niet** nodig: de dump bevat het volledige schema. Controleer tot slot of de applicatie start en of inloggen werkt.

5. Losse database-restore (zonder volledige recovery)

Alleen de gegevens terugzetten (code ongewijzigd)? Pak het archief uit (stap 1) en voer alleen stap 4 uit met `database.sql`.

6. AVG & beveiliging

- Back-ups bevatten alle persoonsgegevens én de encryptiesleutel — uitsluitend versleuteld en intern bewaren; niet e-mailen of naar buiten brengen.
- Echte productiedata alleen in de laatste fase, onder toezicht van de Functionaris Gegevensbescherming.
- Inzage/mutatie van cijfers en BSN wordt gelogd (audit-log, alleen-lezen voor Beheer).
- Rolscheiding wordt server-side afgedwongen; wijzig dit niet zonder reden.
- **Directie is opleidinggebonden.** De koppeltabel `directie_opleidingen` (user ↔ opleiding) bepaalt welke studenten, cijfers en rapporten een directielid ziet. Beheer wijst dit toe via **Gebruikers & rollen → Directie — opleidingtoewijzing**. Zonder toewijzing ziet een directielid **niets** (need-to-know). Een dubbel ingeschreven student is zichtbaar voor de directie van elke opleiding waarin hij/zij actief is. De filtering loopt via `User::opleidingIds()`, `Student::scopeZichtbaarVoor()` en per-opleiding gefilterde statistieken.
- **Presentiegegevens zijn onderwijsinhoudelijk.** Studentenzaken, Financiële Administratie en Beheer hebben géén toegang tot presentielijsten of aanwezigheidspercentages (Gate presentie-inzien). Registreren mag alleen de docent van het eigen vak (Gate presentie-registreren). Inzage en mutatie worden gelogd.

7. Presentie (aanwezigheidsregistratie)

De docent registreert per college de aanwezigheid; dit is verplicht. Het model is bewust **genormaliseerd**: één regel per student × vak × onderwijsweek — nooit vaste weekkolommen op de inschrijving.

Tabel	<code>presenties</code> (<code>inschrijving_id</code> , <code>vak_id</code> , <code>week</code> , <code>aanwezig</code> , <code>geregistreerd_door_id</code>) met unieke sleutel op (<code>inschrijving_id</code> , <code>vak_id</code> , <code>week</code>).
Regeling	Kolom <code>inschrijvingen.aanwezigheidsregeling_50</code> (boolean). Bewust op de inschrijving , niet op de student: zij geldt per opleiding en per studiejaar en moet bij herinschrijving opnieuw worden toegekend.
Normen	<code>config/sis.php</code> → <code>presentie.weken_per_blok</code> (8), <code>presentie.norm</code> (0.80) en <code>presentie.norm_regeling</code> (0.50).
Logica	<code>App\Support\Presentiebewaking</code> — percentage, norm per student, volledigheid per week. <code>App\Support\Statistiek</code> levert de dashboard-aggregaties.

Ontbrekende registratie is geen afwezigheid. Het percentage wordt berekend over de **geregistreerde** weken. Een week zonder regel telt niet mee — anders zou nalatigheid van de docent op de student worden afgewenteld. Een week geldt pas als “geregistreerd” wanneer alle presentieplichtige deelnemers een waarde hebben.

Vrijgestelde studenten (`vaktoewijzingen.vrijgesteld`) volgen het vak niet en worden bij het opslaan overgeslagen, óók als het formulier voor hen een waarde meestuurt. Wijzigt u `weken_per_blok`, dan blijven bestaande registraties met een hoger weeknummer in de database staan maar verdwijnen zij uit het scherm; ruim ze in dat geval expliciet op.

8. Collegegeld: termijnen

Er is bewust **geen facturentabel**. Het termijnschema wordt volledig afgeleid uit het jaartarief, de betaalregeling en de inschrijvingsduur, zodat het nooit kan verouderen ten opzichte van de inschrijving (bijvoorbeeld na een gewijzigde uitschrijfdatum).

Schema	<code>App\Support\Collegegeldtermijnen</code> — vervalmaanden 9, 11, 1, 3, 5; bedrag = $\text{jaarbedrag} \div n$, restje op de laatste termijn.
Regeling	<code>inschrijvingen.betalregeling</code> : termijnen (5 facturen) of volledig (1 factuur, vervalt 1 september).
Betaling → termijn	<code>betalingen.termijn</code> (1..5, nullable). Leeg = automatisch toerekenen aan de oudste openstaande termijn (FIFO).
Achterstand	Som van het openstaande deel van de termijnen waarvan de vervaldatum verstreken is. Dit stuurt de blokkades op herinschrijven en verklaringen.

Per opleiding (beleid 2026-07-10). Collegegeld hangt aan de INSCHRIJVING, niet aan het studiejaar. Elke inschrijving heeft een eigen termijnschema en eigen betalingen; `Collegegeldstatus::voor()` telt alle inschrijvingen op. Korting staat op `inschrijvingen.korting_percentage` (+ verplichte `korting_reden`); `Collegegeldstatus::jaarbedrag()` geeft het tarief ná korting en is de basis voor het schema. Bij 100% korting levert `Collegegeldtermijnen::voor()` een lege collectie. Betalingen worden nooit tussen opleidingen verrekend. Een achterstand bij één inschrijving zet achterstand op true en blokkeert herinschrijven en verklaringen.

Schuld en blokkade zijn twee dingen. `Collegegeldstatus::voor()` geeft achterstand (er is een onbetaalde vervallen termijn) én geblokkeerd (achterstand zonder lopende betalingsafpraak). Blokkeer altijd op `geblokkeerd` / `isGeblokkeerd()`, nooit op achterstand — dat laatste beschrijft alleen

de schuld en blijft true tijdens een afspraak. Tabel betalingsafspraken (geldig_tot, reden, vastgelegd_door_id, ingetrokken_op). 'Lopend' is afgeleid (Betalingsafspraak::isLopend()), nooit een opgeslagen status: een verlopen of ingetrokken afspraak kan zo niet blijven doorwerken. Vastleggen/intrekken alleen door rollen financien en beheerder; beide gelogd (veld: betalingsafspraak).

Betalingen zijn muteerbaar. BetalingController::bijwerken() en ::verwijderen() (rollen financien + beheerder) schrijven de oude en nieuwe waarden naar de audit-log (veld: betaling). Verwijder deze logging niet: zij is het enige bewijsspoor van mutaties op geldbedragen.

Synthetische studenten opschonen. Artisan-commando `php artisan sis:studenten-verwijderen {nummers*} [--force]` verwijdert studenten op studentnummer (los of als reeks, bijv. 261015-261026). Zonder `--force` toont het alleen een voorbeeld (dry-run); met `--force` verwijdert het definitief, in een transactie en per student gelogd (AuditLogger::VERWIJDERING, veld student). Alle gekoppelde gegevens verdwijnen via de database-constraints: inschrijvingen, betalingen, resultaten, presenties, vaktoewijzingen, notities, documenten, vrijstellingsbesluiten, betalingsafspraken en kennistoetsresultaten staan op `ON DELETE CASCADE`; **taken** en **ondertekende documenten** op `SET NULL` (die blijven bestaan, alleen de koppeling vervalst). NB: de studententabel heet studenten (niet students). Uitsluitend bedoeld voor het opschonen van synthetische testdata.

Jaarovergang (actief studiejaar). Het systeem is periode-geparametriseerd: precies één perioden-rij heeft `actief = true`, afgedwongen door het `Periode::saved-event` (activeren van één jaar deactiveert de rest). Alle runtime-logica leidt het jaar af uit de **periode van de inschrijving** of uit `now()` — nergens staat een studiejaar hardcoded in een berekening. Termijndata (Collegegeldtermijnen) komen uit `periode.startdatum`; het tarief uit `collegegeldtarieven` op `periode_id` (opleiding-specifiek vóór een eventueel `opleiding_id = null` standaardtarief van dezelfde periode). De jaarovergang naar 2026-2027 op de draaiende DB is gezet met de guarded datamigratie `activeer_studiejaar_2026_2027` (raw update — een migratie vuurt geen Eloquent-event; daarom expliciet alle jaren op inactief en daarna 2026-2027 op actief). Op een verse migratie draait die vóór de seeders (lege perioden) en doet niets, zodat de ReferentieSeeder-basislijn (2025-2026 actief) en de tests ongewijzigd blijven. Beheer stuurt verdere overgangen via Opzoektabellen → Studiejaren (ReferentieController, checkbox actief). Let op: RapportController::actieveStudentenExport filtert op `status = actief` over ALLE perioden, niet op de actieve periode — bewust, zodat na de overgang zowel herinschreven als nog-niet-herinschreven actieve studenten meekomen.

Let op de betekenis van de velden in `Collegegeldstatus::voor()`: verschuldigd is het totaal van de niet-vervallen termijnen (bij een lopende inschrijving dus het volle jaarbedrag), openstaand is

verschuldigd – betaald (inclusief termijnen die nog moeten vervallen), en achterstallig is het direct opeisbare bedrag. Alleen achterstallig bepaalt achterstand.

De kolom inschrijvingen.betalwijze is **vervallen** (zij mengde regeling en betaalwijze) en blijft alleen voor de historie bestaan. De betaalwijze hoort bij een betaling, niet bij de inschrijving.

Bij een beëindigde inschrijving wordt het totaal herrekend naar het pro rata bedrag; termijnen met een vervaldatum ná het einde worden vervallen en de laatste geldende termijn vangt het verschil op. De CSV-import herkent kolommen op **naam** uit de kopregel, met terugval op de klassieke volgorde (studentnummer;bedrag;datum;betaalwijze;opmerking) voor oudere bestanden.

9. Curriculum (vakken)

Bron	database/data/curriculum.csv (uit 'vakkenlijst update.xlsx', 2026-07-10), geladen door CurriculumSeeder. Referentiedata, geen persoonsgegevens: hoort in Git.
Herladen	php artisan db:seed --class=CurriculumSeeder — idempotent: matcht op (opleiding, code) en werkt bij. Vakken die niet in de CSV staan blijven ongemoeid.
EC	vakken.ec is decimal(4,1): halve studiepunten (2,5) komen voor. Nooit naar int casten. Gebruik App\Support\Ec::toon() voor weergave (Nederlandse komma).
Vakcode	Uniek per opleiding: unique(opleiding_id, code). Elf codes bestaan in zowel ISLTH als PMGV.
Blok	null = het vak loopt het hele studiejaar (stage, scriptie). Views die over blok 1..4 itereren moeten blok null apart tonen.
Keuzevak	vakken.keuzevak. Vaktoewijzer slaat keuzevakken over; Overgangsbeoordeling telt alleen de keuzevakken mee die aan de inschrijving zijn toegewezen.

Synthetische vakken horen niet naast het echte curriculum. De voorbeeldvakken met code ISLTH-* zijn verplaatst naar SynthetischVakSeeder, die alleen door de testsuite wordt gebruikt en bewust NIET in DatabaseSeeder staat. Stonden zij actief naast het echte curriculum, dan telden zij mee in de EC-totalen per leerjaar (jaar 1 werd 94 i.p.v. 60 EC) en werden zij automatisch aan elke ISLTH-student toegewezen.

Nog te doen: de nieuwe vakken hebben **geen docent** gekoppeld (vakken.docent_id is leeg) en krijgen elk één standaard toetsonderdeel ('Tentamen', weging 100%). Koppel de docenten en

verfijn de toetsopbouw via **Vakstructuur** voordat docenten cijfers gaan invoeren; zonder docentkoppeling blijft 'Mijn vakken' leeg.

10. Takenlijst (Studentenzaken)

Tabel taken, model naar Outlook Taken / Microsoft Graph todoTask: titel, omschrijving, startdatum, vervaldatum, status (open | bezig | afgerond), prioriteit (laag | normaal | hoog), afgerond_op, afgerond_door_id. Optionele koppelingen: student_id, toegewezen_aan_id, aangemaakt_door_id, afgerond_door_id — alle vier nullable, zodat een taak blijft bestaan als een student of medewerker verdwijnt.

afgerond_op en afgerond_door_id volgen altijd de status: bij afronden worden beide gezet, bij heropenen beide op null. Dat gebeurt zowel via de afvinkknop als via het bewerkformulier. De afvinker is bewust een eigen veld en niet toegewezen_aan_id: een niet-toegewezen taak mag door iedereen worden opgepakt, en een collega mag andermans taak afronden. Bij taken die vóór deze kolom werden afgerond blijft het veld leeg; dat wordt niet met terugwerkende kracht ingevuld.

'Te laat' is geen kolom. De status wordt afgeleid uit vervaldatum < vandaag én status != afgerond (Taak::isTeLaat()). Sla dit nooit op: anders kan een afgeronde taak in de database als 'te laat' blijven staan.

Toegang uitsluitend voor Studentenzaken en Beheer (Role::magTakenBeheren(), routegroep rol:studentenzaken,beheerder). Er is bewust **geen audit-logging**: een taak is werkverdeling, geen gevoelig persoonsgegeven. Wel blijft zichtbaar wie de taak aanmaakte en aan wie zij is toegewezen.

11. Regulier onderhoud

- **Migraties:** php artisan migrate na een update (reversible; maak eerst een back-up).
- **Cache:** php artisan optimize:clear bij onverwacht gedrag na wijzigingen.
- **Logs:** storage/logs/ (zitten niet in de back-up).
- **Tests:** php artisan test moet groen zijn vóór uitrol.

Deze handleiding wordt bijgewerkt zodra er functies of infrastructuur wijzigen. Controleer de datum onderaan op actualiteit.